

P2953R4

Forbid defaulting operator=(X&&) &&

Published Proposal, 2026-02-22

Authors:

[Matthew Taylor](#)

[Arthur O'Dwyer](#)

Audience:

SG17

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Draft Revision:

12

Abstract

Current C++ permits explicitly-defaulted special members to differ from their implicitly-defaulted counterparts in various ways, including parameter type and ref-qualification. This permits implausible declarations like `A& operator=(const A&) && = default`, where the left-hand operand is rvalue-ref-qualified. We propose to forbid such declarations.

Table of Contents

1 Changelog

2 Motivation and proposal

- 2.1 Interaction with P2952
- 2.2 "Deleted" versus "ill-formed"
- 2.3 Existing corner cases

3 Implementation experience

4 Straw poll results

- 4.1 Proposed polls

5 Proposed wording

- 5.1 [dcl.fct.def.default]

6 Proposed wording (conservative)

- 6.1 [dcl.fct.def.default]

References

Informative References

1. Changelog

- R4:

- Implementation of ambitious approach, deployment considerations.
- R3:
 - Expand motivation: [\[P3834\]](#) also has to solve this problem.
 - Update references to standard versions and features that didn't make C++26.
 - Propose the "ambitious" wording by default.
- R2 (pre-Sofia):
 - Add poll results from the EWG telecon of 2025-01-08.
 - Add electronic poll results after the EWG telecon of 2025-01-08.
 - Update the vendor-divergence table: GCC has fixed [#116162](#).
 - Replace the word "signature" with "declaration" in the motivation section. ("Signature" doesn't include return type.)
 - Add a missing clause to the "ambitious" wording. (Thanks, Jens Maurer.)
- R1:
 - Propose an "ambitious" wording as well as the "conservative" surgery.

§ 2. Motivation and proposal

Currently, [\[dcl.fct.def.default\]/2.5](#) permits an explicitly defaulted special member function to differ from the implicit one by adding *ref-qualifiers*, but not *cv-qualifiers*.

For example, the declaration `const A& operator=(const A&) const& = default` is forbidden because it is additionally const-qualified, and also because its return type differs from the implicitly-defaulted `A&`. This might be considered unfortunate, because it's a reasonable signature for a const-assignable proxy-reference type. But programmers aren't clamoring for that declaration to be supported, so we do not propose it here.

Our concern is that the *unrealistic* declaration `A& operator=(const A&) && = default` is *permitted*! This has several minor drawbacks:

- The possibility of these unrealistic declarations makes C++ harder to understand.
- Additional papers in this space (such as [\[P3834\]](#)) must consider adding new, similarly implausible signatures to the language to remain consistent with this oddity.
- The quirky interaction with [\[CWG2586\]](#) and [\[P2952\]](#) discussed in the next subsection.
- The wording to permit these declarations is at least a tiny bit more complicated than if they weren't permitted.

To eliminate these drawbacks, we propose that an explicitly defaulted copy/move assignment operator should not be allowed to add an *rvalue* ref-qualifier to the type it would have had if implicitly defaulted.

```
struct C {
    C& operator=(const C&) && = default;
    // C++26: Well-formed
    // Proposed: Unusable (deleted or ill-formed)
};

struct D {
    D& operator=(this D&& self, const C&) = default;
    // C++26: Well-formed
    // Proposed: Unusable (deleted or ill-formed)
};
```

This proposal applies only to *explicitly defaulted* `operator=`, and only when the object parameter itself is rvalue-ref-qualified. We propose that it remain legal to write rvalue-ref-qualified `operator=` functions by hand; the compiler simply shouldn't assume it knows how to default one.

We present two options to resolve this issue: Our preferred approach makes many problematic declarations ill-formed; an alternative "conservative" approach makes them defaulted-as-deleted.

When reviewing existing wording in [\[dcl.fct.def.default\]/2.6](#), we noticed that this current wording also permits cv-qualified assignment operators, and move assignment operators which accept a cv-qualified non-object parameter, to be defaulted-as-deleted. These signatures are similarly arcane, we expect that no users make use of them, and these signatures don't help with template programming either (see [§ 2.2 "Deleted" versus "ill-formed"](#)). Therefore we propose making all these signatures ill-formed too.

In summary, our preferred wording makes all of the below declarations ill-formed, while our "conservative" wording makes them all deleted instead:

```
//rvalue-ref-qualified overload (permitted in '26)
foo& operator=(const foo&) && = default;
foo& operator=(foo&&) && = default;

//cv-qualified overload (deleted in '26)
foo& operator=(const foo&) const = default;
foo& operator=(foo&&) const = default;

//cv-qualified rvalue reference parameter (deleted in '26)
foo& operator=(const foo&&) = default;
```

§ 2.1. Interaction with P2952

[\[CWG2586\]](#) (adopted for C++23) permits `operator=` to have an explicit object parameter.

[\[P2952\]](#) (currently in CWG for C++29) proposes that defaulted `operator=` overloads should (also) be allowed to have a placeholder return type. If C++29 gets P2952 without P2953, then we'll have:

```
struct C {
    auto&& operator=(this C&& self, const C&) { return self; }
    // C++26: OK, still deduces C&&

    auto&& operator=(this C&& self, const C&) = default;
    // C++26: Ill-formed, return type contains auto
    // C++29 after P2952: OK, deduces C&
    // Proposed (preferred): Ill-formed, object parameter is not C&
    // Proposed (conservative): Deleted, object parameter is not C&
};
```

The first, non-defaulted, operator "does the natural thing" by returning its left-hand operand, and deduces `C&&`. The second operator also "does the natural thing" by being defaulted; but it deduces `C&`, just like any other defaulted assignment operator. The two "natural" implementations deduce different types! This looks inconsistent.

If we adopt P2953 alongside P2952, then the second `operator=` will go back to being unusable, which reduces the perception of inconsistency.

	C++26	P2952
C++26	<code>C&&/ill-formed</code>	<code>C&&/C&</code>
P2953	<code>C&&/ill-formed</code>	<code>C&&/ill-formed</code>

2.2. "Deleted" versus "ill-formed"

(See also [P2952] §3.2 "Defaulted as deleted".)

[dcl.fct.def.default]/2.6 goes out of its way to make many explicitly defaulted constructors, assignment operators, and comparison operators "defaulted as deleted," rather than ill-formed. This was done by [P0641] (resolving [CWG1331]), in order to support class templates with "canonically spelled" defaulted declarations:

```
struct A {
    // Permitted by (2.4)
    A(A&) = default;
    A& operator=(A&) = default;
};

template<class T>
struct C {
    T t_;
    explicit C();
    // Permitted, but defaulted-as-deleted, by (2.6), since P0641
    C(const C&) = default;
    C& operator=(const C&) = default;
};

C<A> ca; // OK
```

There is similar wording in [class.spaceship] and [class.eq]. We don't want to interfere with these use-cases; that is, we want to continue permitting programmers to write things like the above `C<A>`.

We consider the carve-out for the copy assignment operator of `C<A>` in the above example sensible and do not intend to interfere with it. However, as nobody ever writes `B& operator=(const B&) && = default` we do not need to add any *new* carveouts.

2.3. Existing corner cases

There is vendor divergence in some corner cases. Here is a table of the divergences we found, plus our opinion as to the currently conforming behavior, and our proposed behavior. Red cells in this table indicate non-conformance among vendors today.

URL	Code	Clang	GCC	MSVC	EDG	Correct	Proposed (conservative)
link	<code>C& operator=(C&) = default;</code>	✓	✓	✓	✓	✓	✓
link	<code>C& operator=(const C&&) = default;</code>	deleted	✓	✗	deleted	deleted	deleted
link	<code>C& operator=(const C&) const = default;</code>	deleted	✓	✗	deleted	deleted	deleted
link	<code>C& operator=(const C&) && = default;</code>	✓	✓	✓	✓	✓	deleted
link	<code>C&& operator=(const C&) && = default;</code>	✗	✗	✗	✗	✗	✗
link	<pre>template<class> struct C { static const C& f();</pre>	✓	✗	✗	✗	✓	✓

	<pre>C& operator=(decltype(f()) = default; };</pre>						
link	<pre>struct M { M& operator=(const M&) volatile; }; struct C { volatile M m; C& operator=(const C&) = default; };</pre>	deleted	deleted	deleted	deleted	deleted	deleted
link	<pre>struct A { A& operator=(A&); }; struct C { A a; C& operator=(const C&) = default; };</pre>	deleted	deleted	X	deleted	deleted	deleted
link	<pre>struct A { A& operator=(A&); }; struct C { A a; C& operator=(const C&); }; C& C::operator=(const C&) = default;</pre>	X	X	X	X	X	X

§ 3. Implementation experience

Arthur has implemented both [§ 5 Proposed wording](#) and [§ 6 Proposed wording \(conservative\)](#) in forks of Clang, and used them to compile both LLVM/Clang/libc++ and another large C++17 codebase. Naturally, neither patch caused any problems except in the relevant parts of Clang's own test suite. Matthew has also grepped a large C++20 codebase and found that it does not contain any of the signatures we seek to make ill-formed in [§ 5 Proposed wording](#).

We have also searched for any real-world use of these operator overloads to assess deployment. Searching GitHub for [rvalue-ref-qualified defaulted assignment](#) currently yields around 1.5k results. The vast majority of these are compiler tests and test files from LLVM forks. Refining the search [to remove test files and forks of Clang and LLVM](#) reduces the count down to 6.

Running [a similarly refined search](#) for cv-qualified assignment yields 10 results, all of which comment out the overload in question.

Finally, we ran [a search for cv-qualified parameters in defaulted move assignment](#) and got 36 results.

Note that these searches will undercount slightly, as GitHub search imposes a limit on the greediness of regex used. It is possible that there are additional examples of these signatures on GitHub which use a slightly different series of whitespace, which would not be counted in this search. However, we can get a very approximate measure on this by comparing against [a similarly-constrained search](#) for the "canonical" defaulted assignment operator forms, which yields 311k results. As such, we are confident that the breakage from making these signatures ill-formed would be insignificant.

§ 4. Straw poll results

Arthur O'Dwyer presented P2592R1 (not P2953, but P2952) in the EWG telecon of 2025-01-08. In addition to the vote for forwarding P2952 to CWG, the following straw poll relevant to P2953 was taken. The result was interpreted as "no consensus"; but the numbers (6 for, 1 against) are still a strong signal that EWG was favorably inclined toward P2953 in general.

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>EWG prefers this paper contains the change in P2953 (banning explicitly defaulted operator= with rvalue ref-qualifier). [Chair: This means EWG wants to see this paper again.]</i>	2	4	9	1	0

P2952R1 went to electronic polling, and passed 3–6–1–1–0. The sole voter "Against" P2952's adoption gave as their rationale (paraphrased): "Without P2953 these changes add a corner case to the language. We should prevent that corner case by applying P2953 at the same time as P2952."

4.1. Proposed polls

We suggest the following straw polls.

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>EWG prefers §6 Proposed wording (conservative), which leaves <code>X& operator=(X&&) const = default</code>; and <code>X& operator=(const X&&) = default</code>; defaulted-as-deleted rather than make them ill-formed.</i>	–	–	–	–	–
<i>Advance P2953 to CWG for C++29.</i>	–	–	–	–	–

5. Proposed wording

DRAFTING NOTE: The intent of this "ambitious" wording is to lock down the permissible types of defaultable member functions as much as possible, and make errors as eager as possible, *except* in the cases covered by §2.2 "Deleted" versus "ill-formed", which we want to keep working, i.e., "defaulted as deleted."

DRAFTING NOTE: The only defaultable special member functions are default constructors, copy/move constructors, copy/move assignment operators, and destructors. Of these, only the assignment operators can ever be cvref-qualified.

5.1. [dcl.fct.def.default]

DRAFTING NOTE: The new (2.5) ensures that `struct A { A(A&); }; struct C { A a; C(const C&) = default; };` remains defined-as-deleted, and `struct A { A(A&); }; struct C { A a; C&& operator=(const C&) = default; };` remains ill-formed (not defined-as-deleted, despite that it matches the pattern in (2.5); because it also differs in a second way).

DRAFTING NOTE: Basically all of this wording is concerned specifically with copy/move assignment operators, so it might be nice to move it out of [dcl.fct.def.default] and into [class.copy.assign]. Also note that right now a difference in `noexcept`-ness is handled explicitly by [dcl.fct.def.default] for special member functions but only by omission-and-note in [class.compare] for comparison operators.

Modify [dcl.fct.def.default] as follows:

1. A function definition whose *function-body* is of the form `= default` ; is called an *explicitly-defaulted* definition. A function that is explicitly defaulted shall

- (1.1) be a special member function or a comparison operator function (`{over.binary}`), and
- (1.2) not have default arguments.

2. An explicitly defaulted special member function F_1 is allowed to differ from the corresponding special member function F_2 that would have been implicitly declared, as follows:

- (2.1) ~~if F_2 is an assignment operator, F_1 and F_2 may have differing *ref-qualifiers*; F_1 may have an lvalue *ref-qualifier*;~~
- (2.2) if F_2 ~~has an implicit object parameter of type "reference to C"~~ ~~is an assignment operator with an implicit object parameter of type C&~~, F_1 may ~~be an explicit object member function whose~~ ~~have an~~ explicit object parameter ~~is~~ of type (possibly different) "reference to C" C&, in which case the type of F_1 would differ from the type of F_2 in that the type of F_1 has an additional parameter;
- (2.3) F_1 and F_2 may have differing exception specifications; ~~and~~
- (2.4) if F_2 has a non-object parameter of type `const C&`, the corresponding non-object parameter of F_1 may be of type C&; ~~and~~
- ~~(2.5) if F_2 has a non-object parameter of type C&, the corresponding non-object parameter of F_1 may be of type `const C&`; in this case only, if F_1 is explicitly defaulted on its first declaration then it is defined as deleted; otherwise the program is ill-formed.~~

If the type of F_1 differs from the type of F_2 in a way other than as allowed by the preceding rules, then:

- ~~(2.5) if F_1 is an assignment operator, and the return type of F_1 differs from the return type of F_2 or F_1 's non-object parameter type is not a reference, the program is ill-formed;~~
- ~~(2.6) otherwise, if F_1 is explicitly defaulted on its first declaration, it is defined as deleted;~~
- ~~(2.7) otherwise,~~ the program is ill-formed.

[...]

6. Proposed wording (conservative)

6.1. [decl.fct.def.default]

Modify [decl.fct.def.default] as follows:

1. A function definition whose *function-body* is of the form `= default` ; is called an *explicitly-defaulted* definition. A function that is explicitly defaulted shall

- (1.1) be a special member function or a comparison operator function (`{over.binary}`), and
- (1.2) not have default arguments.

2. An explicitly defaulted special member function F_1 is allowed to differ from the corresponding special member function F_2 that would have been implicitly declared, as follows:

- (2.1) ~~if F_2 is an assignment operator, F_1 and F_2 may have differing *ref-qualifiers*; F_1 may have an lvalue *ref-qualifier*;~~
- (2.2) if F_2 ~~has an implicit object parameter of type "reference to C"~~ ~~is an assignment operator with an implicit object parameter of type C&~~, F_1 may ~~be an explicit object member function whose~~ ~~have an~~ explicit object parameter ~~is~~ of type (possibly different) "reference to C" C&, in which case the type of F_1 would differ from the type of F_2 in that the type of F_1 has an additional parameter;
- (2.3) F_1 and F_2 may have differing exception specifications; and
- (2.4) if F_2 has a non-object parameter of type `const C&`, the corresponding non-object parameter of F_1 may be of type C&.

If the type of F_1 differs from the type of F_2 in a way other than as allowed by the preceding rules, then:

- (2.5) if ~~F_1~~ E_2 is an assignment operator, and the return type of F_1 differs from the return type of F_2 or F_1 's non-object parameter type is not a reference, the program is ill-formed;
- (2.6) otherwise, if F_1 is explicitly defaulted on its first declaration, it is defined as deleted;
- (2.7) otherwise, the program is ill-formed.

[...]

References

Informative References

[CWG1331]

Daniel Krügler. *const mismatch with defaulted copy constructor*. June 2011. URL: <https://cplusplus.github.io/CWG/issues/1331.html>

[CWG2586]

Barry Revzin. *Explicit object parameter for assignment and comparison*. May–July 2022. URL: <https://cplusplus.github.io/CWG/issues/2586.html>

[P0641]

Daniel Krügler; Botond Ballo. *Resolving CWG1331: const mismatch with defaulted copy constructor*. November 2017. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0641r2.html>

[P2952]

Arthur O'Dwyer; Matthew Taylor. *auto& operator=(X&&) = default*. August 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2952r0.html>

[P3834]

Matthew Taylor; Alex (Waffl3x); Oliver Rosten. *Defaulting the Compound Assignment Operators*. October 2025. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3834r1.html>